

Web Application Development

IS4102

Lecture Note 09 - XML

K T Dananjali
Lecturer (Unconfirmed)
Department of Computing and Information Systems
Faculty of Computing

Overview: XML

- XML stands for eXtensible Markup Language.
- HTML and XML look similar, because they are both SGML
- languages (SGML = Standard Generalized Markup Language)
 - Both HTML and XML use elements enclosed in tags (e.g.
 - `<body>This is an element</body>`)
 - Both use tag attributes (e.g.,
 - `<font face="Verdana" size="+1" color="red">`)
 - Both use entities (<, >, &, ", ')

Overview: XML

- XML is a markup language developed for the web, and it differs from traditional scripting and programming languages in its purpose and structure.
- XML is not mainly used for processing or displaying data. Instead, it helps computers understand what the data represents.
- There are two main reasons for the development of XML:
 - Computers do not understand the information placed in them.
 - Web pages are not compatible across different devices.

Features and Advantages of XML

- XML separates data from HTML

If you need to display dynamic data in your HTML document, then the data can be stored in separate XML files. With a few lines of JavaScript code, you can read an external XML file and update the data content of your web page.

- XML simplifies data sharing

Computer systems and databases contain data in incompatible formats. XML data is stored in plain text format. This provides a software- and hardware-independent way of storing data.

Features and Advantages of XML

- XML simplifies data transport

Exchanging data as XML greatly reduces this complexity, since the data can be read by different incompatible applications.

- XML simplifies Platform change

It easier to expand or upgrade to new operating systems, new applications, or new browsers, without losing data.

Features and Advantages of XML

- XML increases data availability

With XML, your data can be available to all kinds of "reading machines" (Handheld computers, voice machines, news feeds, etc), and make it more available for blind people, or people with other disabilities.

- XML can be used to create new internet languages

XHTML/ WSDL/ RDF/ OWL/ WAP/ WML

XML vs HTML

HTML	XML
HTML is used to mark up text so it can be displayed to users.	XML is used to mark up data so it can be processed by computers.
HTML describes both structure (e.g. <code><p></code> , <code><h2></code> , <code></code>) and appearance (e.g. <code>
</code> , <code></code> , <code><i></code>)	XML describes only content, or “meaning”
HTML uses a fixed, unchangeable set of tags	In XML, you make up your own tags

XML Vs HTML

<h1> Bibliography </h1>

<p> <i> Foundations of DBs</i>, Abiteboul, Hull, Vianu

 Addison-Wesley, 1995

<p> <i> Logics for DBs and ISs </i>, Chomicki, Saake, eds.

 Kluwer, 1998

<bibliography>

<book> <title> Foundations of DBs </title>

<author> Abiteboul </author>

<author> Hull </author>

<author> Vianu </author>

<publisher> Addison-Wesley </publisher>

....

.</book>

<book> ... <editor> Chomicki </editor>... </book> ...

</bibliography>

HTML tags:

*presentation, generic
document structure*

XML tags:

*content, "semantic",
(DTD-) specific*

XML-Related Technologies

- DTD (Document Type Definition) and XML Schemas are used to define legal XML tags and their attributes for particular purposes
- CSS (Cascading Style Sheets) describe how to display HTML or XML in a browser.
- XSLT (eXtensible Stylesheet Language Transformations) and XPath are used to translate from one form of XML to another.

XML Syntax

XML tags

- Tags are formed by a word (or a number of words) enclosed inside `<>` and `</>` signs. Just like, for example the `<b></b>` tag in HTML.
- XML tags are not pre-defined like HTML. XML tag `<message>` and the end tag `</message>` which could be used to enclose an e-mail message stored on a web based e-mail system.

XML Syntax

XML declaration

`<?xml version="1.0" encoding="UTF-8" standalone="yes"?>`

- The XML declaration must be first--not even whitespace should precede it
- Note that the brackets are `<?` and `?>`
- `version="1.0"` is required
- `encoding` can be `"UTF-8"` (ASCII) or `"UTF-16"` (Unicode), or something else, or it can be omitted
- `standalone` tells whether there is a separate DTD

XML Syntax

XML Text

➤ XML has only one “basic” type – text

➤ It is bounded by tags, e.g.,

`<title> The Big Sleep </title>`

`<year> 1935 </ year>` – 1935 is still text

XML Syntax

The Structure of XML

Tags must be properly nested

➤ good

```
<date> ... <day> ... </day> ... </date>
```

➤ bad

```
<date> ... <day> ... </date>... </day>
```

XML Syntax

XML Structure

➤ Nesting tags can be used to express various structures, e.g., a tuple (record):

<person>

 <name> Tharidu</name>

 <tel> 071-xxxxxxx </tel>

 <email> tharidu@gmail.com </email>

</person>

XML Syntax

XML Structure

- We can represent a list by using the same tag repeatedly:

<addresses>

 <person> ... </person>

 <person> ... </person>

 <person> ... </person>

 <person> ... </person>

...

</addresses>

Syntax

XML Structure

<addresses>

 <person>

 <name> Banujan</name>

 <tel> 078-xxxxxxxxxx</tel>

 <email> banu@gmail.com </email>

 </person>

 <person>

 <name> Supun</name>

 <tel> 071-xxxxxxxx </tel>

 <email> supu@yahoo.com </email>

 </person>

</addresses>

XML Building blocks

Element

- Delimited by angle brackets
- Identify the nature of the content they surround
- General format: `<element> ... </element>`
- Empty element: `</empty-Element>`

XML Building Blocks

Attribute

- Name-value pairs that occur inside start-tags after element name, like: `<element attribute="value">`
- Located in the start tag of elements
- Provide additional information about elements
- Often provide information that is not a part of data
- Must be enclosed in quotes.

Elements and their Content

element type

element

~~<bibliography>~~

element
content

<paper ID="11111">

<authors>

<author>S.Kumara</author>

<author>I.Paik</author>

<author>W.Chen</author>

</authors>

<fullPaper source="JWSR"/>

<title>Service Clustering</title>

<booktitle> WSC </booktitle>

</paper>

empty element

</bibliography>

character content

Element Attributes

Attribute name

Attribute Value

<bibliography>

<paper **pid**="11111">

<authors>

<author>S. Kumara</author>

<author>I. Paik</author>

<author>W. Chen</author>

</authors>

<fullPaper **source**="JWSR"/>

<title>Service clustering</title>

<booktitle>WSC</booktitle>

</paper>

</bibliography>

XML Building blocks

Entities

- Five special characters must be written as entities:
 - & for & (almost always necessary)
 - < for < (almost always necessary)
 - > for > (not usually necessary)
 - " for " (necessary inside double quotes)
 - ' for ' (necessary inside single quotes)
- These entities can be used even in places where they are not absolutely required.
- These are the only predefined entities in XML.

Names in XML

- Names (as used for tags and attributes) must begin with a letter or underscore, and can consist of:
 - Letters, both Roman (English) and foreign
 - Digits, both Roman and foreign
 - . (dot)
 - - (hyphen)
 - _ (underscore)
 - : (colon) should be used only for namespaces

XML Comments

`<!-- This is a comment in both HTML and XML -->`

- Comments can be put anywhere in an XML document.
- Comments are useful for:

Explaining the structure of an XML document

Commenting out parts of the XML during development and testing

- Comments are not displayed by browsers, but can be seen by anyone who looks at the source code.
- These are the only predefined entities in XML.

XML

`<?xml version="1.0" encoding="UTF-8"?>`

- - `<note>`
 - `<to>Tove</to>`
 - `<from>Jani</from>`
 - `<heading>Reminder</heading>`
 - `<body>Don't forget me this weekend!</body>`
- `</note>`

<?xml version="1.0" encoding="UTF-8"?>

<CATALOG>

<CD title="Empire Burlesque">

<ARTIST>Bob Dylan</ARTIST>

<COUNTRY>USA</COUNTRY>

<PRICE>10.90</PRICE>

<YEAR>1985</YEAR>

</CD>

<CD title="Hide your heart">

<ARTIST>Bonnie Tyler</ARTIST>

<COUNTRY>UK</COUNTRY>

<PRICE>9.90</PRICE>

<YEAR>1988</YEAR> </CD>

<cd>.....</cd>

<cd>.....</cd>

<cd>.....</cd>

<cd>.....</cd>

</CATALOG>

Example

Reg No.	Name	Marks
001	Kirihami	32
002	Hinmanike	66
003	Appuhami	85
004	Sirimali	45
005	Harry	99

Identifying Vocabularies: XML Namespaces

- My element may not be your element:
 - geometry context: `<element>line</element>`
 - chemistry context: `<element>oxygen</element>`
- use XML namespaces to identify the vocabulary
- An XML document may reference more than one DTD
- Namespaces are a way to specify which DTD defines a given tag.

Identifying Vocabularies: XML Namespaces

- A namespace is defined as a unique string
 - To guarantee uniqueness, typically a URI (Uniform Resource Indicator) is used, because the author “owns” the domain
 - It doesn't have to be a “real” URI; it just has to be a unique string
- There are two ways to use namespaces:
 - Declare a default namespace
 - Associate a prefix with a namespace, then use the prefix in the XML to refer to the namespace

Default Namespace

```
➤ <table xmlns="http://www.w3.org/TR/html4/">  
  <tr>  
    <td>Apples</td>  
    <td>Bananas</td>  
  </tr>  
</table>
```

Associate a prefix with a namespace

```
<root  
  xmlns:h="http://www.w3.org/TR/html4/"  
  xmlns:f="http://www.w3schools.com/furniture">
```

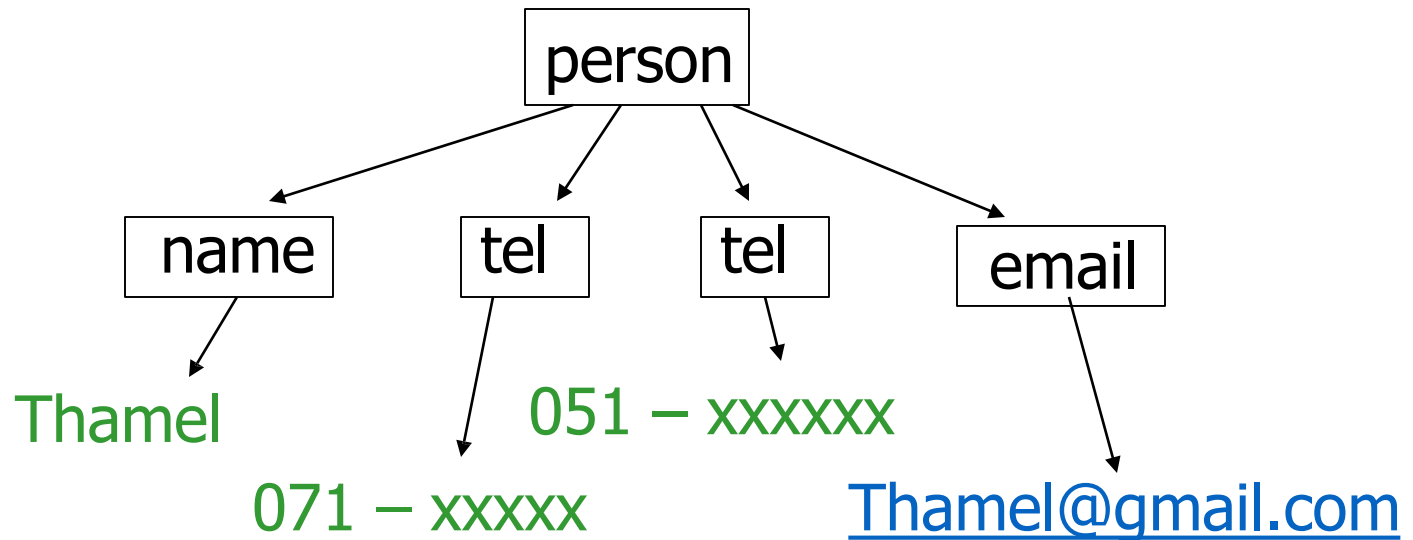
```
  <h:table>  
    <h:tr>  
      <h:td>Apples</h:td>  
      <h:td>Bananas</h:td>  
    </h:tr>  
  </h:table>
```

```
  <f:table>  
    <f:name>African Coffee Table</f:name>  
    <f:width>80</f:width>  
    <f:length>120</f:length>  
  </f:table>  
</root>
```

Well-Formed XML Documents

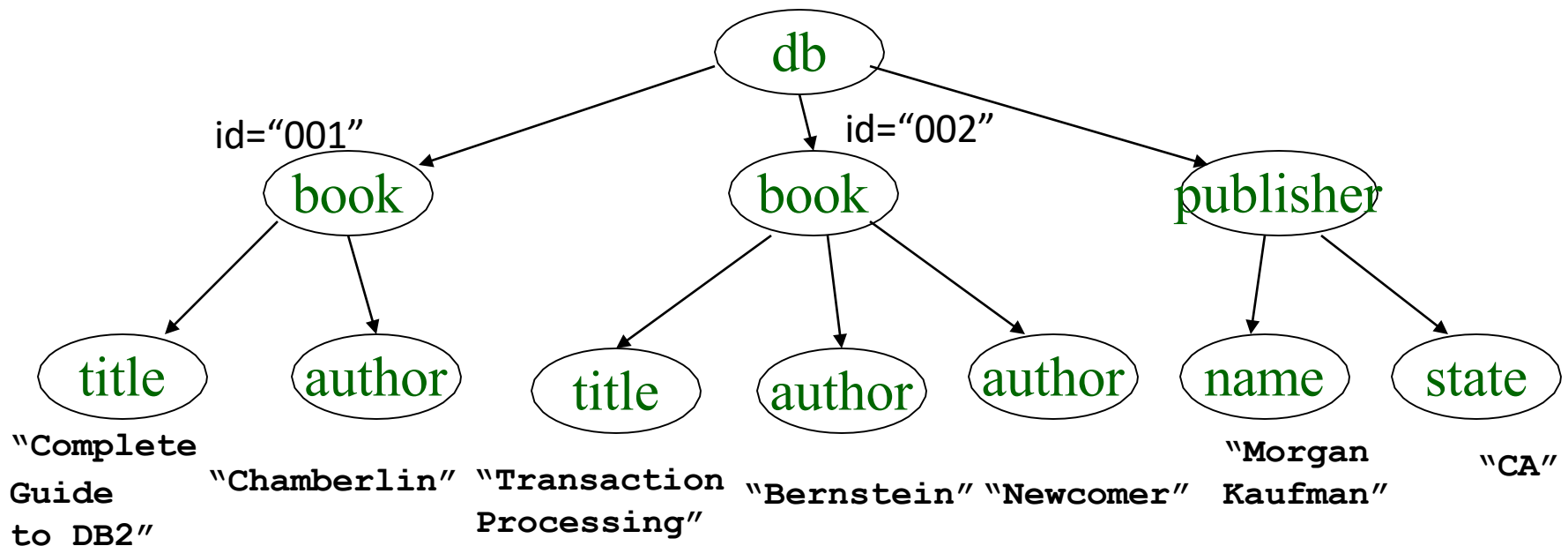
- An XML document (with or without a DTD) is well-formed if
 - Tags are syntactically correct
 - Every tag has an end tag
 - Tags are properly nested
 - There is a root tag

XML as a Data Model (Tree)



Leaves are either empty or contain DATA

XML as a Data Model (Tree)



Tags on nodes

Data values on leaves

example

<bank>

 <account account_number="A-401">

 <branch_name> Downtown </branch_name>

 <balance> 500 </balance>

 </account>

 <customer customer_id="C100" >

 <customer_name>Joe </customer_name>

 <customer_street> Monroe </customer_street>

 <customer_city> Madison</customer_city>

 </customer>

 <customer customer_id="C102">

 <customer_name> Mary </customer_name>

 <customer_street> Erin </customer_street>

 <customer_city> Newark </customer_city>

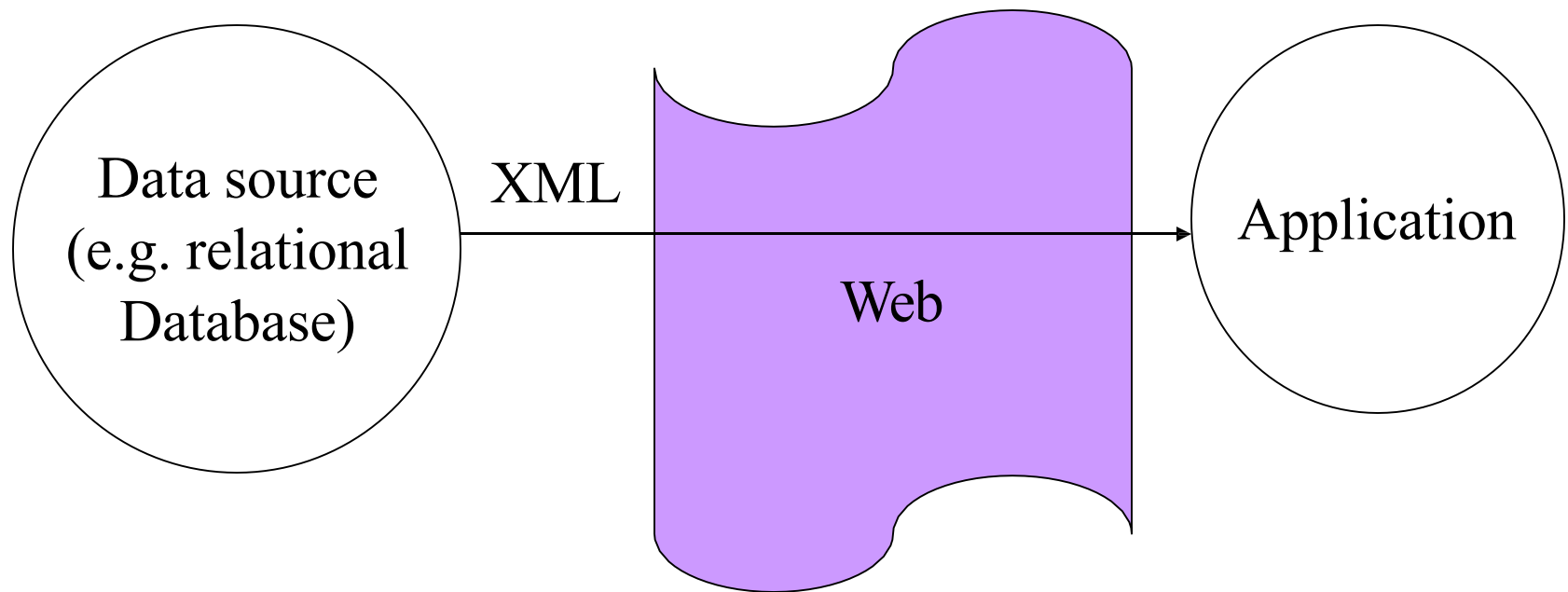
 </customer>

</bank>

```
<imdb>
  <show year="1993">
    <title>AAAAA</title>
    <review>
      <suntimes>
        <reviewer>Roger Ebert</reviewer> <rating>two thumbs
          up</rating>
      </suntimes>
    </review>
  </show>
  <review>
    <nyt>The standard &hollywood; summer movie strikes back.</nyt>
  </review>
  <box_office>183,752,965</box_office>
</show>
<show year="1994">
  <title>X Files,The</title>
  <seasons>4</seasons>
</show>
</imdb>
```

Exporting Relational Data to XML

Data Sharing with XML:



Representing relational databases

A relational database for school:

student:

id	name	gpa
001	Joe	3.0
002	Mary	4.0
...	...	...

course:

cno	title	credit
331	DB	3.0
350	Web	3.0
...	...	...

enroll:

id	cno
001	331
001	350
002	331
...	...

XML representation

```
<school>
  <student id="001">
    <name> Joe </name>
    <gpa> 3.0 </gpa>
  </student>
  <student id="002">
    <name> Mary </name>
    <gpa> 4.0 </gpa>
  </student>
  <course cno="331">
    <title> DB </title>
    <credit> 3.0 </credit>
  </course>
  <course cno="350">
    <title> Web </title>
    <credit> 3.0 </credit>
  </course>
```

XML representation

```
<enroll>  
    <id> 001 </id>  
    <cno> 331 </cno>  
</enroll>  
<enroll>  
    <id> 001 </id>  
    <cno> 350 </cno>  
</enroll>  
<enroll>  
    <id> 002 </id>  
    <cno> 331 </cno>  
</enroll>  
</school>
```


XML to relational database

```
<bank>
  <account account_number="A-401" owners="C100 C102">
    <branch_name> Matara </branch_name>
    <balance>      500000 </balance>
  </account>
  <customer customer_id="C100" accounts="A-401">
    <customer_name>Omal </customer_name>
    <customer_street> Colombo street </customer_street>
    <customer_city>   Battaramulla</customer_city>
  </customer>
  <customer customer_id="C102" accounts="A-401 A-402">
    <customer_name>Sashini </customer_name>
    <customer_street> 5th Street </customer_street>
    <customer_city>   Galle </customer_city>
  </customer>
</bank>
```